



Mekelle University

Mekelle Institute of Technology(MIT)

Department of information technology (IT)

ML Group Assignment

No.	Group members name	ID NO.
1	Abraham Michael	MIT/UR/004/12
2	Alazar Goitom	MIT/UR/007/12
3	Florida H/slassie	MIT/UR/043/12
4	Kidan G/aregawi	MIT/UR/083/12
5	Mahlet Ezra	MIT/UR/215/12
6	Netsanet Meaza	MIT/UR/113/12
7	Rshan G/medhin	MIT/UR/249/12
8	Senait G/hans	MIT/UR/218/12

Mushroom Classification Report

A. Introduction

Objective:

The objective of this analysis is to predict whether a mushroom is edible or poisonous using a dataset containing various features describing the mushroom's characteristics. This is a binary classification problem where we aim to use machine learning models to predict the class of a mushroom based on its properties.

Importance of solving this problem:

Identifying edible and poisonous mushrooms is critical for safety, especially for people who forage mushrooms. A model trained on a reliable dataset can help to quickly classify mushrooms, making it easier to distinguish between the two classes and prevent potential poisoning from consuming toxic mushrooms.

B. Data Exploration (EDA)

Dataset Description:

The dataset contains 22 features, including the class of the mushroom (edible or poisonous) and attributes related to the mushroom's physical characteristics (e.g., shape, surface, color, odor, gill attachment, and habitat). The target variable is class, which is binary with values:

- 0: Edible
- 1: Poisonous

The features in the dataset are categorical and need to be converted to numeric form for machine learning algorithms to process them.

Key Visualizations:

- **Histograms:**

The histograms display the distribution of each feature in the dataset. These visualizations help identify if any feature has an imbalanced distribution or if it follows a specific pattern.

- **Correlation Matrix:**

The correlation matrix is used to check the relationships between numerical features. It helps identify any strong correlations that could influence the model's performance and feature selection.

- **Pairplot:**

The pairplot visualization explores the relationships between a selected subset of features (such as class, odor, gill-color, spore-print-color, habitat). It visualizes how the features interact with each other and their potential impact on the target variable.

C. Methodology

Preprocessing Steps:

1. **Missing Value Handling:**

The dataset is checked for missing values. In case of any, rows containing missing values are removed. This ensures that the model is trained on complete data without missing entries.

2. **Label Encoding:**

All categorical variables are encoded using LabelEncoder, transforming each unique category in the columns into numeric values. This is necessary because machine learning algorithms require numeric inputs.

3. **Feature Selection:**

The target variable class is separated from the feature columns, and the features are prepared for model training.

Algorithms Chosen and Reasoning:

1. **Decision Tree Classifier:**

Decision trees are chosen due to their simplicity and interpretability. They are useful for classification problems with categorical data and can easily handle non-linear relationships between features.

2. Random Forest Classifier:

Random forests are chosen as an ensemble learning method. They aggregate multiple decision trees to improve the performance by reducing overfitting and increasing the accuracy compared to a single decision tree.

D. Model Training

Training Details:

- **Hyperparameters:**

- For the DecisionTreeClassifier, the default hyperparameters are used, with the random state set for reproducibility.
- For the RandomForestClassifier, 100 trees are used (`n_estimators=100`) to create a robust model that reduces overfitting and increases predictive performance.

- **Validation Strategy:**

The dataset is split into training and testing sets using an 80/20 split (`train_test_split` with `test_size=0.2`), where 80% of the data is used for training and 20% is used for testing. This ensures that the model is evaluated on unseen data, allowing us to assess its generalization capabilities.

E. Results and Evaluation

Evaluation Metrics:

The performance of both models is evaluated using the following metrics:

- **Accuracy:** Proportion of correctly classified instances.
- **Precision:** Proportion of true positive predictions among all positive predictions.
- **Recall:** Proportion of true positive predictions among all actual positive instances.
- **F1 Score:** Harmonic mean of precision and recall, providing a balanced evaluation metric.

Confusion Matrix:

A confusion matrix is used to summarize the results of classification. It shows

the true positives, true negatives, false positives, and false negatives, helping to visualize how well the models perform in terms of both types of errors.

Performance Interpretation: The model's performance is interpreted by analyzing the confusion matrix and the classification report, which provides detailed insights into each evaluation metric. The higher the accuracy, precision, recall, and F1 score, the better the model's ability to distinguish between the classes.

F. Conclusion

Summary of Findings:

Both the **Decision Tree** and **Random Forest** models are trained and evaluated for predicting whether a mushroom is edible or poisonous. The Random Forest model is expected to perform better than the Decision Tree due to its ensemble nature, which reduces overfitting and variance.

Challenges Faced:

- Handling missing values in the dataset was essential to ensure the quality of the data before training the models.
- Label encoding of categorical features may result in a loss of information, as some algorithms might interpret encoded numeric values as having an ordinal relationship, even though they don't.

Possible Improvements:

- **Feature Engineering:** Further feature engineering could be applied, such as creating new features or using domain knowledge to improve the model's performance.
- **Hyperparameter Tuning:** Further optimization of hyperparameters (e.g., for decision tree depth, random forest number of trees, etc.) could lead to better model performance.
- **Cross-Validation:** Using k-fold cross-validation would provide more robust estimates of the model's performance by reducing variability caused by random splits in the data.

G. Appendix:

=====

1. Import Libraries

```
# =====
```

```
import warnings
```

```
warnings.filterwarnings("ignore")
```

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import LabelEncoder
```

```
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.ensemble import RandomForestClassifier
```

```
from sklearn.metrics import accuracy_score, precision_score, recall_score,  
f1_score, classification_report, confusion_matrix
```

```
# Set plot style
```

```
sns.set(style='whitegrid')
```

```
# =====
```

2. Load Dataset

```
# =====
```

```
# Replace with the dataset path or link
```

```
df = pd.read_csv('/kaggle/input/mushromdataset/agaricus-lepiota.data',  
header=None)
```

```
columns = [
```

```
    'class', 'cap-shape', 'cap-surface', 'cap-color', 'bruises', 'odor',
```

```
    'gill-attachment', 'gill-spacing', 'gill-size', 'gill-color', 'stalk-shape',
```

```
'stalk-root', 'stalk-surface-above-ring', 'stalk-surface-below-ring',  
'stalk-color-above-ring', 'stalk-color-below-ring', 'veil-type', 'veil-color',  
'ring-number', 'ring-type', 'spore-print-color', 'population', 'habitat'  
]
```

```
df.columns = columns
```

```
# =====
```

3. Exploratory Data Analysis (EDA)

```
# =====
```

```
print(df.info()) # Check for column types
```

```
print(df.describe()) # Get summary statistics
```

```
print(df.isnull().sum()) # Check for missing values
```

```
# Handle missing data if any
```

```
df.replace('?', np.nan, inplace=True)
```

```
df.dropna(inplace=True) # Drop rows with missing values
```

```
# Ensure all columns are numeric for pairplot
```

```
# After label encoding, make sure the 'class' column is also encoded as  
numeric
```

```
label_encoders = {}
```

```
for col in df.columns:
```

```
    le = LabelEncoder()
```

```
    df[col] = le.fit_transform(df[col])
```

```
    label_encoders[col] = le
```

```
# Check if all columns are now numeric
```

```
print(df.dtypes)
```

```
# Visualizations
```

```
df.replace('?', np.nan, inplace=True)
```

```
df.dropna(inplace=True) # Ensure no missing values after replacing '?' with NaN
```

```
# Ensure no infinite values are in the dataset
```

```
df.replace([np.inf, -np.inf], np.nan, inplace=True)
```

```
df.dropna(inplace=True) # Drop rows with any NaN values after replacing infinities
```

```
label_encoders = {}
```

```
for col in df.columns:
```

```
    le = LabelEncoder()
```

```
    df[col] = le.fit_transform(df[col])
```

```
    label_encoders[col] = le
```

```
X = df.drop('class', axis=1)
```

```
y = df['class']
```

```
# =====
```

```
# Histograms
```

```
# =====
```

```
df.hist(bins=20, figsize=(18, 14), color='skyblue', edgecolor='black')
```

```
plt.suptitle("Feature Histograms", fontsize=20)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# =====  
  
# Correlation Matrix  
  
# =====  
  
plt.figure(figsize=(16, 12))  
correlation = df.corr()  
sns.heatmap(correlation, annot=True, cmap="coolwarm", linewidths=0.5)  
plt.title("Correlation Matrix Heatmap", fontsize=16)  
plt.show()
```

```
# =====  
  
# Pairplot of Selected Features  
  
# =====  
  
selected_features = ['class', 'odor', 'gill-color', 'spore-print-color', 'habitat']  
sns.pairplot(df[selected_features], hue='class', palette='husl')  
plt.suptitle("Pairplot of Selected Features", y=1.02)  
plt.show()
```

```
# =====
```

4. Preprocessing

```
# =====
```

```
# Encode categorical variables
```

```
label_encoders = {}
```

```
for col in df.columns:
```

```
    le = LabelEncoder()
```

```
    df[col] = le.fit_transform(df[col])
```

```
    label_encoders[col] = le
```

```
X = df.drop('class', axis=1)
```

```
y = df['class']
```

```
# =====
```

5. Train/Test Split

```
# =====
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,  
random_state=42)
```

```
# =====
```

6. Model Training

```
# =====
```

```
# Decision Tree Classifier
```

```
dt_model = DecisionTreeClassifier(random_state=42)
```

```
dt_model.fit(X_train, y_train)
```

```
# Random Forest Classifier
```

```
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
```

```
rf_model.fit(X_train, y_train)
```

```
# =====
```

7. Prediction & Evaluation

```
# =====
```

```
# Evaluation function
```

```
def evaluate_model(name, y_true, y_pred):
```

```
    print(f"--- {name} ---")
```

```
print("Accuracy:", accuracy_score(y_true, y_pred))  
print("Precision:", precision_score(y_true, y_pred))  
print("Recall:", recall_score(y_true, y_pred))  
print("F1 Score:", f1_score(y_true, y_pred))  
print("Confusion Matrix:\n", confusion_matrix(y_true, y_pred))  
print(classification_report(y_true, y_pred))
```

Evaluate models

```
evaluate_model("Decision Tree", y_test, dt_model.predict(X_test))  
evaluate_model("Random Forest", y_test, rf_model.predict(X_test))
```

kaggle link:- <https://www.kaggle.com/code/alazargoitom/mushroom-classification>